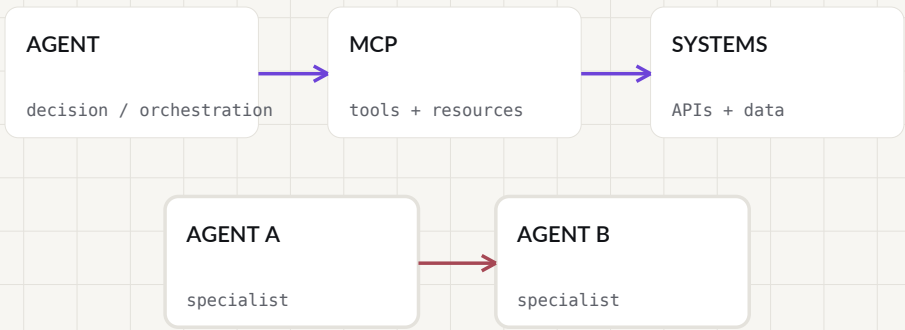


MCP + A2A

THE INTEROPERABILITY LAYER FOR AGENTIC AI

Two different boundaries. One production architecture: agents need a standard way to reach capabilities - and a standard way to collaborate with other agents.



MCP = agent to capability / A2A = agent to agent

PROBLEM

Agents do not fail only because models fail

The integration layer becomes the bottleneck when every agent invents its own tool

BEFORE

Custom tool wrappers Custom auth
rules Custom discovery Custom
agent handoffs

**WITH STANDARDS**

Shared capability interface Shared
discovery semantics Explicit task
handoffs Observable boundaries

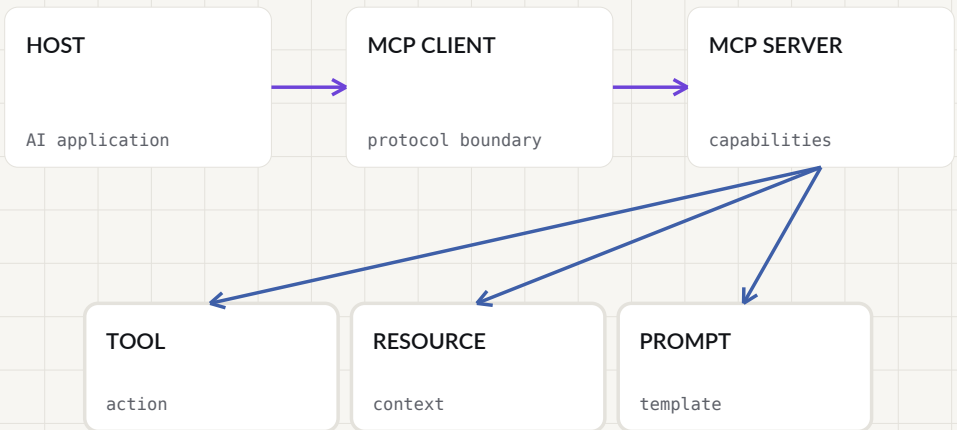
THE IMPORTANT DISTINCTION

MCP and A2A solve different interoperability problems. Treating them as competing protocols creates the wrong architecture.

MCP

Model Context Protocol

A standardized interface for an AI application to discover and use external capabilities such as tools and resources.



MENTAL MODEL

The agent should not care how every CRM, database, search system or internal API is wired. The capability boundary owns that complexity.

LIFECYCLE

One tool call, end to end

Keep the boundary explicit so permissions, validation, timeouts and observability sit where they belong.

01 User request

02 Agent decides

03 MCP client

04 MCP server

05 Tool / resource

06 Result

PRODUCTION CHECKS

Schema validation Authorization
before execution Timeout +
bounded retries Structured errors
Trace ID across the call

BOUNDARY

MCP is not “an API that an LLM can call”

The useful abstraction is the standardized interaction boundary: discovery, capability description, invocation and context exchange.

TRADITIONAL API

MCP

Endpoint / route

Capability / tool

Human-designed contract

Model-consumable schema

App owns discovery

Protocol exposes capabilities

Custom integration logic

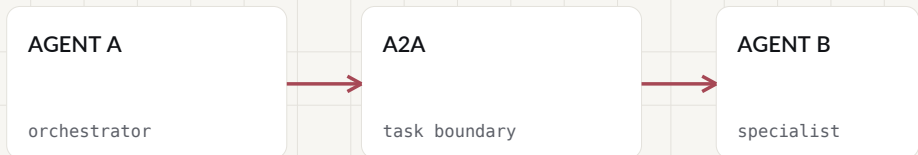
Standardized client-server boundary

MCP does not replace APIs. It is an interoperability layer that lets AI applications interact with capabilities without rebuilding every integration pattern from scratch.

A2A

Agent-to-agent collaboration

A2A addresses the other boundary: one agent delegates or collaborates with another specialized agent.

**TASK**

A2A is useful when the receiving side owns meaningful autonomy: its own tools, context, policy or execution strategy.

EXAMPLE

Research Agent → Compliance Agent → Finance Agent. Each specialist owns its own domain boundary; the caller cares about the task and result, not the internal implementation.

LIFECYCLE

Task handoff, not just a function call

The in 01 Discover / address the agent action".

01

Discover / address the agent

**02**

Send task + context

**03**

Receive status / artifacts

**04**

Complete / resume / escalate

WHY THIS MATTERS

Long-running work needs explicit status, cancellation, retry and artifact handling.

MCP VS A2A

Different problems. Different boundaries.

Do not choose by popularity. Choose by the boundary your system needs to

AXIS	MCP	A2A
Connects	Agent → capability	Agent → agent
Unit	Tool / resource	Task / collaboration
Owns	External integration	Agent autonomy
Example	CRM lookup	Risk analyst agent

RULE OF THUMB

Need a capability? Start with MCP. Need another autonomous specialist? Consider A2A. Need both? Compose them.

COMPOSE

MCP and A2A belong in the same system

ORCHESTRATOR

planning / routing

An orchestrator can collaborate with specialist agents while each specialist reaches systems through its own MCP boundary.

RESEARCH AGENT

A2A

FINANCE AGENT

A2A

SEARCH

MCP

DOCS

MCP

ERP

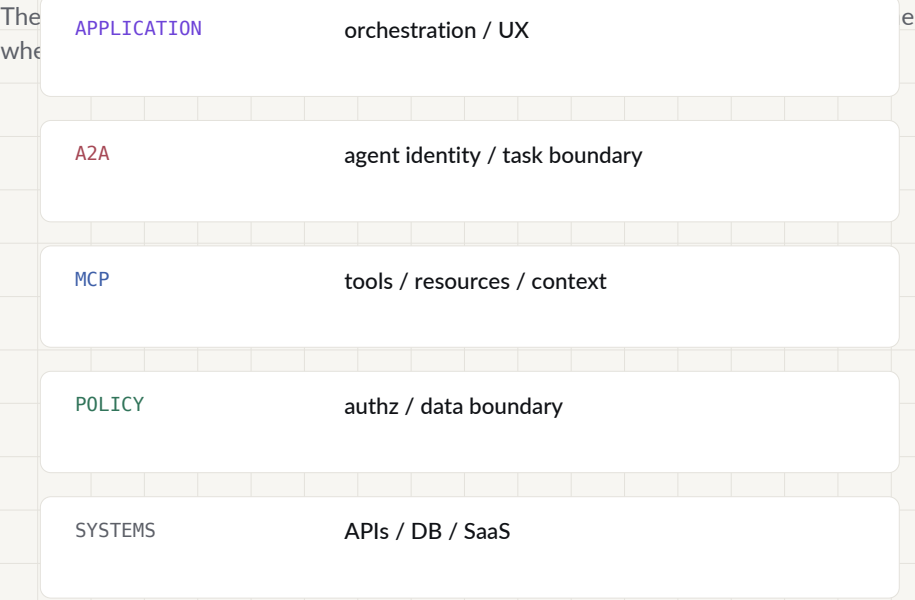
MCP

KEY DESIGN IDEA

A2A handles collaboration across agent boundaries. MCP handles access inside those boundaries.

STACK

Treat interoperability as an architecture layer



The reliable system is not “MCP enabled” or “A2A enabled”. It has explicit boundaries for who can invoke what, on behalf of whom, with which data and for how long.

SECURITY

Every interoperability boundary is an authorization boundary

A tool call or agent handoff can cross trust zones. Treat the boundary as security

if ~~cross-trust-zone~~ ~~interoperability~~

MCP RISKS

Over-broad tools Hidden side effects
Prompt-influenced arguments Data leakage

A2A RISKS

Agent impersonation Untrusted instructions Cross-agent data leakage
Runaway delegation

MINIMUM CONTROLS

Identity + authentication Least-privilege authorization Input / output validation Audit logs
Timeouts and bounded delegation

DESIGN

A good MCP server exposes capabilities, not chaos

The server boundary should make the safe path the easy path.

SCHEMA	Strict input + output types
AUTHZ	Check identity and permissions
VALIDATION	Reject unsafe or impossible requests
ERRORS	Return structured, actionable failures
OBSERVABILITY	Trace every invocation

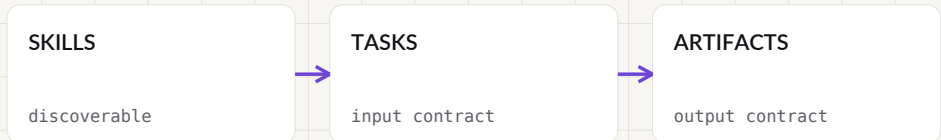
REACH FOR IT, OR DO NOT

Do not wrap every helper function as a tool. Prefer stable capabilities with meaningful domain boundaries and clear side effects.

DESIGN

A2A agents need explicit contracts

A specialist agent should publish what it can do, what it needs, and what it returns - without leaking its internal reasoning or implementation.



HANDOFF CONTRACT

Define ownership, expected completion state, failure semantics, authentication context and the data permitted to cross the boundary.

The caller should not depend on undocumented internal steps. It should depend on the agent capability and task contract.

FAILURES

Three layers can fail independently

MCP transport

request timeout / disconnect
bounded retry + circuit breaker

Tool execution

API rejects / partial side effect
idempotency + compensation

A2A task

agent unavailable / bad artifact
status + resume / fallback

Agent reasoning

wrong plan / wrong tool choice
evaluation + guardrails

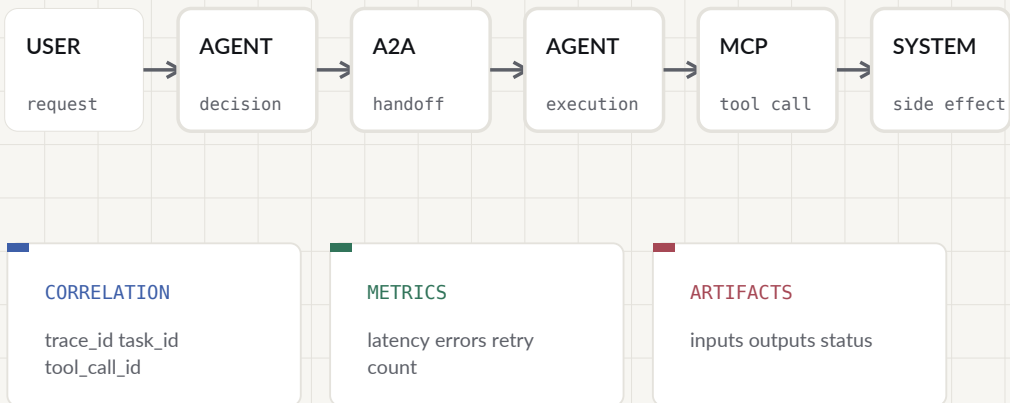
DO NOT DO THIS

Blindly retrying an agent or tool call can duplicate side effects. Retry policy belongs to the operation semantics, not to hope.

TRACE

Trace the whole decision path

A production trace should connect the user request to agent decisions, A2A handoffs, MCP calls and external effects.



Without cross-layer correlation, “the agent was slow” is not a diagnosis. You need to know which handoff or capability caused the delay.

DO I NEED AN AGENT?

Not every capability deserves an agent

Use the smallest abstraction that matches the problem. Deterministic work is often

Is the task deterministic?

→ **Tool / API**

Does it need independent context?

→ **Specialist agent**

Does it need planning across steps?

→ **Agent / workflow**

Does another agent own the domain?

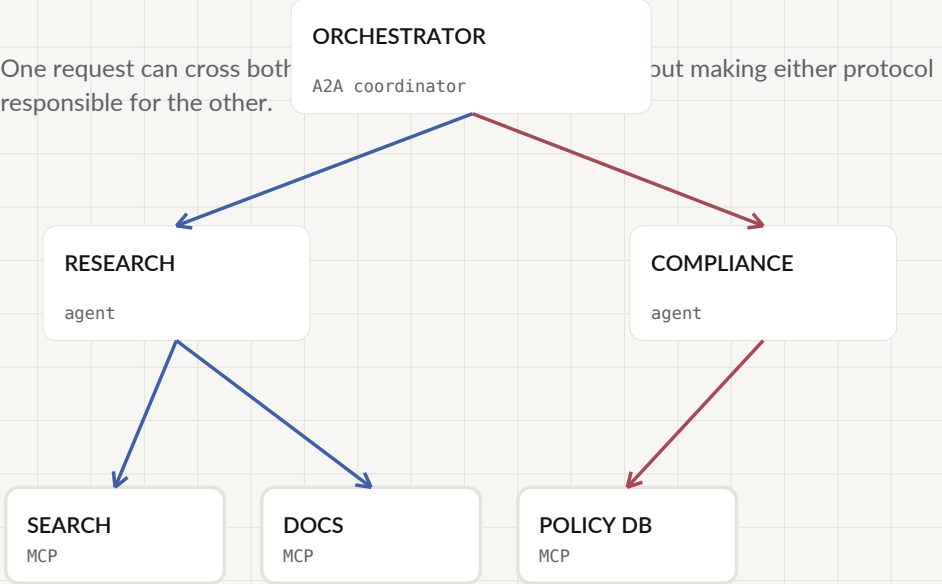
→ **A2A handoff**

DESIGN PRINCIPLE

Adding an agent adds state, policy, observability and failure modes. Pay that cost only when autonomy provides real value.

REFERENCE FLOW

A realistic multi-agent workflow



BOUNDARY LOGIC

A2A carries the collaboration. MCP carries the capability access. The orchestrator owns policy about when a handoff is justified.

PRODUCTION

Before shipping interoperability

A clean protocol diagram is not enough. Check the operational contract around it.

MCP

Identity is explicit Permissions are least-privilege Tool schemas are strict External side effects are understood Retries are bounded

A2A

A2A tasks have ownership Artifacts have clear contracts Timeouts are enforced End-to-end traces exist Failure states are testable

THE SHORT VERSION

Design the protocol boundary first. Then design identity, policy, resilience and observability around it.

CHOOSE

A simple decision map

Start with the interaction

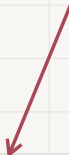
NEED TO USE A CAPABILITY?

protocol name.



MCP

agent ↔ tools / data



NEED ANOTHER SPECIALIST?



NEED BOTH?

A2A

agent ↔ agent

MCP + A2A

compose both

Keep the boundaries orthogonal. That makes systems easier to evolve, secure and observe.

THE BOTTOM LINE

Two protocols. Two boundaries. One architecture.

MCP standardizes how an AI application reaches capabilities. A2A standardizes how agents collaborate across domain boundaries.

MCP → capability access

A2A → agent collaboration

TOGETHER → composable agent systems

ONE LAST RULE

Do not add protocols for the diagram. Add them when they remove a real integration problem - and then make the boundary observable, authorized and failure-aware.